INSTITUTO SUPERIOR TECNOLÓGICO QUITO

Excelencia en Educación Superior

1.1 Neuronas Convolucionales

python™

Erick Mejia

Neuronales Convolucionales (CNN)

Esta investigación proporciona una visión completa de las Redes Neuronales Convolucionales (CNN), incluyendo su definición, historia, funcionamiento interno, aplicaciones prácticas, ventajas y desventajas. Se incluyen ejemplos de código, visualizaciones y un análisis detallado para comprender su importancia en el campo del aprendizaje profundo.

Índice

1. Introducción
2. ¿Qué son las CNN?
3. Historia y Evolución
4. Arquitectura y Funcionamiento
5. Aplicaciones
6. Ventajas y Desventajas
7. Ejemplo Práctico con Código
8. Conclusiones
9. Recomendaciones

10. Referencias

1. Introducción

Las Redes Neuronales Convolucionales (CNN, por sus siglas en inglés Convolutional Neural Networks) representan uno de los avances más significativos en el campo del aprendizaje profundo (deep learning) y la inteligencia artificial. Inspiradas en el funcionamiento del córtex visual de los mamíferos, estas redes están especialmente diseñadas para procesar datos que tienen una estructura de cuadrícula, como imágenes digitales, pero también se han extendido a otras modalidades como señales de audio, texto y datos secuenciales.

El cerebro humano procesa la información visual de manera jerárquica, comenzando con características simples como bordes y texturas, y avanzando hacia conceptos más complejos como formas y objetos. Las CNN imitan este proceso mediante capas que extraen características progresivamente más abstractas. Esta inspiración biológica ha sido clave para su éxito en tareas que antes eran difíciles para las máquinas.

Origen y Motivación

El procesamiento de imágenes ha sido históricamente un desafío para las computadoras debido a la alta dimensionalidad y variabilidad de los datos visuales. Las imágenes contienen millones de píxeles, y pequeñas variaciones como rotaciones, escalas o iluminación pueden cambiar drásticamente su representación. Las redes neuronales tradicionales (como los perceptrones multicapa) no eran eficientes para manejar esta complejidad porque trataban cada píxel como una característica independiente, perdiendo las relaciones espaciales locales.

Por ejemplo, en una imagen de 1000x1000 píxeles, una red tradicional requeriría millones de parámetros solo para la primera capa, lo que la hace impráctica. Las CNN resuelven este problema mediante operaciones de convolución que capturan patrones locales en los datos, permitiendo que la red aprenda características jerárquicas: desde bordes simples en las primeras capas hasta conceptos complejos como rostros o objetos en capas más profundas.

Esta eficiencia no solo reduce el número de parámetros, sino que también hace que las CNN sean más robustas a variaciones en la entrada, un aspecto crucial para aplicaciones del mundo real.

Importancia Actual

En la era actual, las CNN son la base de muchas tecnologías de IA que usamos diariamente:

- **Reconocimiento facial** en smartphones y sistemas de seguridad, donde algoritmos como FaceID de Apple utilizan CNN para identificar usuarios con alta precisión.

- **Clasificación de imágenes** en aplicaciones médicas para diagnóstico, permitiendo detectar anomalías en radiografías o resonancias magnéticas.
- **Vehículos autónomos** para detección de peatones y señales de tráfico, esencial para la seguridad vial.
- **Procesamiento de lenguaje natural** a través de arquitecturas como CNN para texto, aunque menos comunes que los Transformers, siguen siendo útiles en ciertas tareas.
- **Generación de imágenes** con GANs (Redes Generativas Antagónicas), donde las CNN generan imágenes realistas para arte, diseño y simulación.

Según estudios recientes, las CNN han logrado superar el rendimiento humano en tareas de clasificación de imágenes en datasets como ImageNet, con accuracies superiores al 95%. Esto ha abierto nuevas posibilidades en campos como la robótica, la medicina y el entretenimiento.

Además, las CNN han impulsado el desarrollo de otras áreas del deep learning, como el procesamiento de video, el análisis de señales biomédicas y la síntesis de audio. Su versatilidad las hace indispensables en la caja de herramientas de cualquier científico de datos o ingeniero de IA.

Estructura de Esta Investigación

Esta investigación está organizada de la siguiente manera:

- **Capítulo 2**: Definición y conceptos básicos de las CNN, incluyendo sus componentes principales y principios de funcionamiento.
- **Capítulo 3**: Historia y evolución de las CNN, desde sus orígenes hasta las arquitecturas modernas.
- **Capítulo 4**: Arquitectura detallada y funcionamiento matemático, con explicaciones de convolución, pooling y funciones de activación.
- **Capítulo 5**: Aplicaciones prácticas en diversos campos, con ejemplos concretos y casos de uso.
- **Capítulo 6**: Ventajas, desventajas y limitaciones, analizando su idoneidad para diferentes escenarios.
- **Capítulo 7**: Ejemplo práctico con implementación en código, usando TensorFlow/Keras para demostrar su uso.
- **Capítulo 8**: Conclusiones que resumen los puntos clave.
- **Capítulo 9**: Recomendaciones para investigadores y practicantes.
- **Capítulo 10**: Referencias bibliográficas que sustentan la investigación.

Cada sección incluye explicaciones teóricas, ejemplos prácticos y referencias a la literatura relevante. Se incluyen visualizaciones y diagramas para facilitar la comprensión, así como código ejecutable para que el lector pueda experimentar.

Metodología

Esta investigación se basa en una revisión exhaustiva de la literatura académica, desde los trabajos pioneros de Yann LeCun en la década de 1980 hasta las arquitecturas modernas como ResNet, EfficientNet y Vision Transformers. Se han consultado papers en conferencias como NeurIPS, CVPR e ICML, así como libros de referencia como "Deep Learning" de Goodfellow et al.

Se incluyen implementaciones prácticas usando frameworks de deep learning como TensorFlow y Keras, con ejemplos ejecutables que permiten al lector reproducir los resultados. Los datasets utilizados, como MNIST y CIFAR-10, son estándares en la comunidad y permiten comparaciones objetivas.

La metodología combina análisis teórico con experimentación práctica, asegurando que las explicaciones sean tanto rigurosas como accesibles. Se han incluido ecuaciones matemáticas donde es necesario, pero siempre con explicaciones intuitivas para no abrumar al lector principiante.

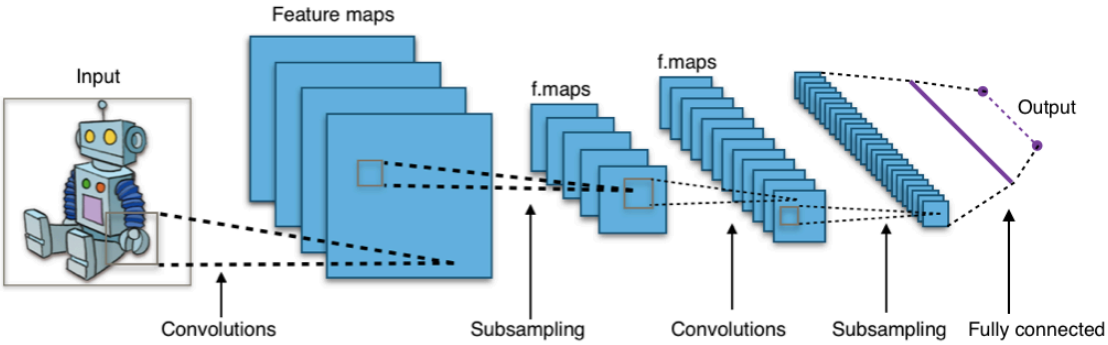


Figura 1: Arquitectura típica de una Red Neuronal Convolucional. Fuente: Wikipedia

Esta figura muestra las capas principales: convolución, pooling, y capas completamente conectadas. La convolución extrae características locales, el pooling reduce la dimensionalidad, y las capas densas realizan la clasificación final.

2. ¿Qué son las CNN?

Las Redes Neuronales Convolucionales son un tipo especializado de red neuronal feedforward diseñada para procesar datos estructurados en cuadrículas. A diferencia de las redes neuronales tradicionales que procesan vectores de características, las CNN operan directamente sobre matrices de datos, preservando la información espacial.

Esta preservación de la estructura espacial es crucial porque permite que la red aprenda patrones que dependen de la ubicación relativa de los píxeles. Por ejemplo, en una imagen, la relación entre píxeles adyacentes es más importante que su valor absoluto.

Componentes Principales

Las CNN están compuestas por varios tipos de capas que trabajan en conjunto para extraer y procesar características:

- **Capas Convolucionales**: Estas son el corazón de la CNN. Aplican filtros (también llamados kernels) que se deslizan sobre la imagen de entrada, computando productos punto en cada posición. Cada filtro está diseñado para detectar características específicas, como bordes horizontales, verticales o diagonales. Los filtros se aprenden durante el entrenamiento, lo que permite que la red descubra automáticamente las características más relevantes para la tarea.

- **Capas de Pooling**: Después de las capas convolucionales, se utilizan capas de pooling para reducir la dimensionalidad de los mapas de características. Esto no solo reduce el número de parámetros y el costo computacional, sino que también hace que la red sea más robusta a pequeñas variaciones en la posición de las características. El max pooling, por ejemplo, selecciona el valor máximo en una ventana pequeña, preservando la característica más prominente.

- **Capas Completamente Conectadas**: Al final de la red, se utilizan capas densas (fully connected) para combinar las características extraídas y realizar la clasificación final. Estas capas funcionan como una red neuronal tradicional, mapeando las características a las clases de salida.

- **Funciones de Activación**: Se aplican después de cada capa convolucional y completamente conectada para introducir no-linealidad. La función ReLU (Rectified Linear Unit) es la más común, ya que acelera el entrenamiento y evita el problema del vanishing gradient.

Además de estos componentes básicos, las CNN modernas incluyen técnicas como dropout para regularización, batch normalization para estabilizar el entrenamiento, y conexiones residuales en arquitecturas como ResNet.

Principios de Funcionamiento

Las CNN se basan en tres principios fundamentales que las hacen efectivas para el procesamiento de imágenes:

1. **Invarianza a la traslación**: Gracias a la operación de convolución, las CNN pueden reconocer patrones independientemente de su posición en la imagen. Si un objeto aparece en la esquina superior izquierda o en el centro, la red lo detectará de manera similar.

2. **Compartición de parámetros**: Los mismos filtros se aplican en toda la imagen, reduciendo drásticamente el número de parámetros a aprender. Esto no solo hace que las CNN sean más eficientes, sino que también las hace más generalizables.

3. **Jerarquía de características**: Las capas iniciales aprenden características de bajo nivel como bordes y texturas, mientras que las capas posteriores combinan estas características en conceptos más abstractos como formas y objetos. Esta jerarquía refleja cómo el cerebro humano procesa la información visual.

Comparación con Redes Tradicionales

A diferencia de las redes neuronales feedforward tradicionales, que requieren que los datos de entrada se aplanen en un vector unidimensional, las CNN mantienen la estructura 2D (o 3D para videos) de los datos. Esto permite capturar relaciones espaciales que serían perdidas en el aplanamiento.

Por ejemplo, en una red tradicional, dos píxeles adyacentes en una imagen se tratarían como características independientes, sin considerar su proximidad. En una CNN, estos píxeles se procesan juntos a través de operaciones locales.

Esta capacidad para manejar datos estructurados ha hecho que las CNN sean la arquitectura de elección para tareas de visión por computadora, y su éxito ha inspirado extensiones a otros dominios como el procesamiento de secuencias y grafos.

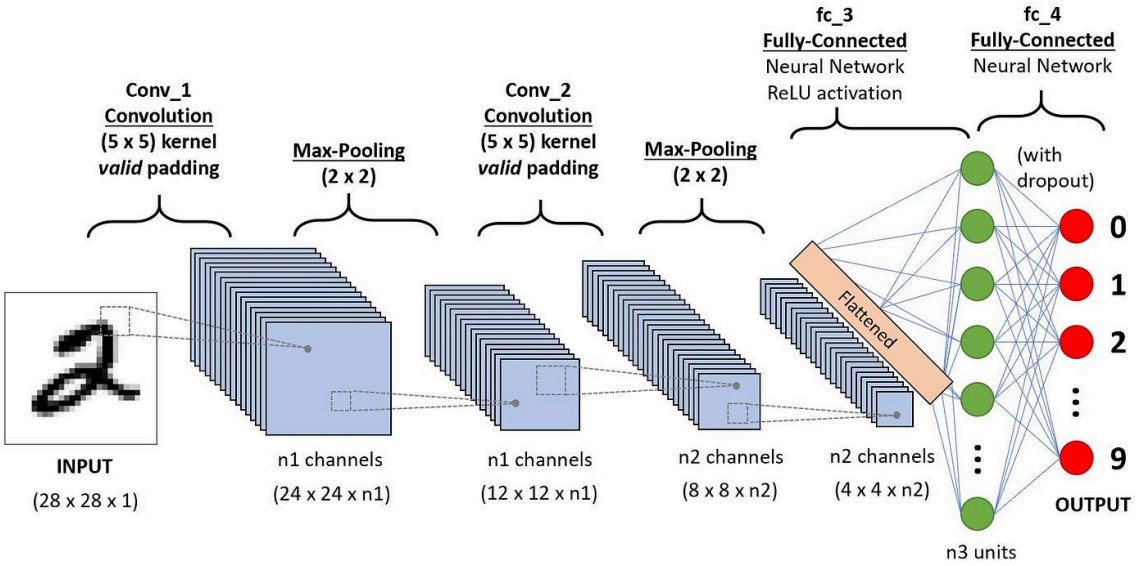


Figura 2: Componentes principales de una CNN. La imagen muestra cómo los datos fluyen desde la entrada a través de capas convolucionales, pooling y completamente conectadas hasta la salida de clasificación.

3. Historia y Evolución

La historia de las CNN comienza en la década de 1980 con el trabajo pionero de Yann LeCun y sus colaboradores en Bell Labs. Aunque los conceptos matemáticos de convolución datan

de mucho antes, fue LeCun quien los aplicó por primera vez a redes neuronales para el reconocimiento de patrones.

Línea de Tiempo Detallada

- **1960s-1970s**: Desarrollo de conceptos básicos de redes neuronales y perceptrones. Investigadores como Marvin Minsky y Seymour Papert critican las limitaciones de los perceptrones para tareas complejas.
- **1980**: Kunihiko Fukushima propone el Neocognitron, una red inspirada en el córtex visual que utiliza operaciones similares a la convolución. Aunque no usa aprendizaje supervisado, sienta las bases para las CNN modernas.
- **1986**: David Rumelhart, Geoffrey Hinton y Ronald Williams publican el algoritmo de backpropagation, que permite entrenar redes profundas de manera eficiente.
- **1989**: Yann LeCun presenta LeNet-1, la primera CNN verdadera entrenada con backpropagation. Diseñada para reconocer dígitos manuscritos, logra accuracies del 80% en el dataset MNIST.
- **1990s**: Desarrollo de LeNet-4 y LeNet-5. En 1998, LeNet-5 se implementa en lectores de cheques en bancos, marcando una de las primeras aplicaciones comerciales de CNN.
- **2000s**: Las CNN caen en relativo olvido debido a limitaciones computacionales y la dominancia de métodos tradicionales como SVM. Sin embargo, investigadores continúan refinando las técnicas.
- **2012**: Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton presentan AlexNet en la competencia ImageNet. Utilizando GPUs para entrenamiento, AlexNet reduce el error de clasificación del 26% al 15.3%, revolucionando el campo del deep learning.
- **2014**: Karen Simonyan y Andrew Zisserman introducen VGGNet, que demuestra que redes más profundas (hasta 19 capas) pueden lograr mejores resultados. VGGNet establece el patrón de arquitecturas profundas con filtros pequeños (3x3).
- **2015**: Kaiming He et al. presentan ResNet (Residual Networks), que resuelve el problema del vanishing gradient mediante conexiones residuales. ResNet permite entrenar redes con cientos de capas, alcanzando accuracies superiores al 96% en ImageNet.
- **2016-2017**: Surgimiento de arquitecturas optimizadas como DenseNet (con conexiones densas entre capas) y EfficientNet (que escala automáticamente la arquitectura).
- **2018-2020**: Integración de atención y transformers en visión. Vision Transformer (ViT) de Google demuestra que transformers pueden competir con CNN en tareas de visión.

- **2021-presente**: Desarrollo de arquitecturas híbridas que combinan lo mejor de CNN y transformers. Avances en eficiencia computacional con técnicas como quantization y pruning.

Factores del Resurgimiento en 2012

El renacimiento de las CNN en 2012 no fue accidental. Varios factores convergieron:

- **Poder Computacional**: La disponibilidad de GPUs baratas (como las de NVIDIA) permitió entrenar modelos grandes en tiempos razonables. AlexNet se entrenó en dos semanas usando dos GPUs GTX 580.
- **Datasets Grandes**: ImageNet, con 1.2 millones de imágenes etiquetadas en 1000 clases, proporcionó los datos necesarios para entrenar modelos profundos sin overfitting.
- **Mejores Algoritmos**: Técnicas como dropout, batch normalization y optimizadores avanzados (Adam) mejoraron la estabilidad y velocidad del entrenamiento.
- **Regularización**: Métodos como data augmentation (rotaciones, recortes) aumentaron efectivamente el tamaño del dataset.

Este resurgimiento no solo revitalizó las CNN, sino que impulsó todo el campo del deep learning, llevando a aplicaciones en medicina, automoción y más.

Impacto en la Comunidad Científica

Las CNN han transformado la investigación en visión por computadora. Antes de 2012, los métodos tradicionales dominaban; después, las CNN se convirtieron en el estándar. Esto ha llevado a una explosión de papers, con conferencias como CVPR y ICCV dedicando secciones enteras a arquitecturas basadas en CNN.

Hoy, las CNN son la base de la visión por computadora moderna, pero también han influido en otros campos como el NLP (con CNN para texto) y el procesamiento de señales. Su evolución continúa, con investigadores explorando cómo integrarlas con otras técnicas de IA.

4. Arquitectura y Funcionamiento

La arquitectura de una CNN típica consiste en una secuencia de capas que transforman la entrada (generalmente una imagen) en una salida (como una etiqueta de clase). Cada capa realiza una operación específica que extrae o procesa características de manera jerárquica.

Operación de Convolución

La convolución es el corazón de las CNN. Un filtro (kernel) se desliza sobre la imagen de entrada, computando el producto punto en cada posición para generar un mapa de características.

Matemáticamente, para una imagen de entrada I y un kernel K, la convolución se define como:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Donde (i,j) es la posición en el mapa de salida, y m,n recorren el kernel. Si el kernel es de tamaño 3x3, se suman 9 productos.

Esta operación captura patrones locales: un kernel con valores altos en el centro y bajos en los bordes detectará manchas brillantes; uno con positivos a la izquierda y negativos a la derecha detectará bordes verticales.

En la práctica, una capa convolucional aplica múltiples filtros en paralelo, produciendo múltiples mapas de características. Por ejemplo, la primera capa de AlexNet usa 96 filtros de 11x11.

Capas de Pooling

Después de la convolución, se aplican capas de pooling para reducir la dimensionalidad y hacer la red más robusta.

- **Max Pooling**: Divide el mapa de características en ventanas no superpuestas (ej. 2x2) y toma el valor máximo en cada ventana. Esto preserva las características más prominentes mientras reduce el tamaño.
- **Average Pooling**: Toma el promedio en lugar del máximo, pero es menos común ya que el max pooling suele dar mejores resultados.

El pooling reduce el número de parámetros y computaciones en capas posteriores, y proporciona invarianza a pequeñas traslaciones.

Funciones de Activación

Las funciones de activación introducen no-linealidad, permitiendo que la red aprenda funciones complejas.

- **ReLU (Rectified Linear Unit)**: f(x) = max(0,x). Es la más popular porque acelera el entrenamiento (evita vanishing gradient) y es computacionalmente eficiente. Sin embargo, puede causar "neuronas muertas" si muchos valores son negativos.
- **Leaky ReLU**: f(x) = max(0.01x, x), que permite un pequeño gradiente para valores negativos.

- **Sigmoid y Tanh**: Usadas en capas de salida para clasificación binaria o multiclase, pero menos comunes en capas intermedias debido al vanishing gradient.

Capas Completamente Conectadas

Al final de la red, las características extraídas se aplanan y pasan por capas densas que realizan la clasificación. Estas capas combinan todas las características aprendidas para producir probabilidades de clase.

Por ejemplo, en una CNN para clasificación de 10 clases, la última capa tendría 10 neuronas con activación softmax.

Entrenamiento

El entrenamiento de CNN usa backpropagation con descenso de gradiente. Los parámetros (pesos de filtros y biases) se actualizan para minimizar una función de pérdida, como cross-entropy para clasificación.

Optimizadores comunes:

- **SGD (Stochastic Gradient Descent)**: Simple pero puede ser lento.
- **Adam**: Combina momentum y adaptive learning rates, convergiendo más rápido.

Durante el entrenamiento, se usan técnicas como:

- **Data Augmentation**: Rotaciones, recortes, flips para aumentar el dataset.
- **Regularización**: Dropout (apaga neuronas aleatoriamente) y L2 regularization para prevenir overfitting.
- **Batch Normalization**: Normaliza las activaciones para estabilizar el entrenamiento.

Arquitecturas Modernas

- **ResNet**: Usa conexiones residuales (skip connections) para permitir gradientes fluyan directamente, resolviendo vanishing gradient.
- **DenseNet**: Conecta cada capa a todas las subsiguientes, maximizando el flujo de información.
- **EfficientNet**: Escala automáticamente profundidad, ancho y resolución para optimizar eficiencia.

Estas arquitecturas han empujado los límites de lo posible, con redes de miles de capas entrenadas exitosamente.

5. Aplicaciones

Las CNN han encontrado aplicaciones en prácticamente todos los campos que involucran procesamiento de datos estructurados. Su capacidad para aprender características automáticamente las hace ideales para tareas donde la ingeniería manual de características sería prohibitiva.

Visión por Computadora

La visión por computadora es el dominio original y más maduro para las CNN:

- **Clasificación de imágenes**: Identificar la clase principal de una imagen. Modelos como ResNet-50 logran accuracies superiores al 95% en ImageNet. Usos incluyen etiquetado automático de fotos en redes sociales.
- **Detección de objetos**: Localizar y clasificar múltiples objetos en una imagen. Algoritmos como YOLO (You Only Look Once) procesan imágenes en tiempo real, crucial para aplicaciones de seguridad y robótica.
- **Segmentación semántica**: Asignar una etiqueta a cada píxel, dividiendo la imagen en regiones significativas. Usado en medicina para segmentar órganos o en agricultura para identificar cultivos.
- **Reconocimiento facial**: Verificar identidades mediante comparación de características faciales. Sistemas como FaceNet utilizan CNN para extraer embeddings que se comparan con bases de datos.

Estas aplicaciones han revolucionado industrias como el comercio electrónico (búsqueda visual) y la seguridad (vigilancia inteligente).

Medicina

Las CNN están transformando el diagnóstico médico al asistir a profesionales con análisis precisos:

- **Diagnóstico de cáncer**: Analizan imágenes histológicas o mamografías para detectar tumores. Estudios muestran que CNN pueden igualar o superar el rendimiento de radiólogos en algunos casos.
- **Análisis de radiografías**: Detectan neumonía, fracturas o anomalías en rayos X. Durante la pandemia de COVID-19, modelos basados en CNN ayudaron a triar pacientes.
- **Segmentación de órganos**: Delimitan órganos en imágenes de resonancia magnética o tomografías, facilitando tratamientos quirúrgicos planificados o radioterapia.

La FDA ha aprobado varios sistemas de IA basados en CNN para uso clínico, marcando un hito en la adopción de deep learning en medicina.

Automoción

La industria automotriz depende de CNN para la seguridad y autonomía:

- **Vehículos autónomos**: Detectan peatones, vehículos y señales de tráfico en tiempo real. Tesla y Waymo utilizan CNN en sus sistemas de visión.
- **Asistencia al conductor**: Monitorean el estado del conductor (fatiga, distracción) y el entorno, activando alertas o interviniendo automáticamente.

Las CNN permiten que los vehículos "vean" y tomen decisiones basadas en datos visuales, un paso crucial hacia la movilidad autónoma.

Otras Áreas

Las CNN se han extendido más allá de las imágenes tradicionales:

- **Procesamiento de lenguaje**: Aunque los transformers dominan, CNN como TextCNN se usan para clasificación de texto, capturando patrones locales en secuencias.
- **Análisis de señales**: Procesan datos de audio para reconocimiento de voz (aunque RNN son más comunes) o señales sísmicas para predicción de terremotos.
- **Generación de imágenes**: En GANs, las CNN generan imágenes realistas. Aplicaciones incluyen arte generativo, diseño de moda y síntesis de datos para entrenamiento.
- **Juegos y entretenimiento**: En juegos como AlphaGo, CNN analizan tableros; en realidad virtual, generan entornos.

Incluso en campos no tradicionales como finanzas (análisis de patrones en gráficos) o biología (predicción de estructuras proteicas), las CNN han encontrado nichos.

Métricas de Rendimiento

Las CNN han logrado accuracies superiores al 99% en datasets como MNIST, pero el rendimiento varía por tarea. En clasificación médica, sensitivities del 95% son comunes, pero siempre se evalúa contra expertos humanos.

El impacto económico es masivo: se estima que las aplicaciones de visión por computadora generarán billones en valor en las próximas décadas.

6. Ventajas y Desventajas

Las CNN ofrecen una combinación única de poder y eficiencia, pero no están exentas de limitaciones. Entender sus fortalezas y debilidades es crucial para elegir cuándo usarlas.

Ventajas

- **Eficiencia en parámetros**: La compartición de parámetros reduce drásticamente el número de pesos a aprender. Una CNN típica tiene millones de veces menos parámetros que una red densa equivalente, haciendo posible entrenar modelos grandes.
- **Invarianza a traslaciones**: Las CNN reconocen patrones independientemente de su posición, lo que las hace robustas a variaciones en la entrada. Esto es invaluable en aplicaciones del mundo real donde los objetos pueden aparecer en cualquier lugar.
- **Jerarquía de características**: Aprenden automáticamente desde características simples (bordes) hasta complejas (objetos), eliminando la necesidad de diseño manual de características. Esto acelera el desarrollo y mejora la precisión.
- **Paralelizable**: Las operaciones convolucionales se prestan bien al paralelismo de GPUs, permitiendo entrenamiento rápido. Frameworks como TensorFlow optimizan automáticamente para hardware paralelo.
- **Versatilidad**: Aunque diseñadas para imágenes, se adaptan a otros datos estructurados como audio (espectrogramas) o texto (matrices de embeddings).

Estas ventajas explican por qué las CNN dominan en visión por computadora, superando métodos tradicionales en la mayoría de benchmarks.

Desventajas

- **Requerimientos de datos**: Necesitan grandes datasets etiquetados para entrenar bien. En dominios con datos limitados, como medicina especializada, esto puede ser un obstáculo.
- **Costo computacional**: El entrenamiento requiere hardware potente (GPUs) y tiempo. Una sola epoch en ImageNet puede tomar horas, y el costo energético es significativo.
- **Interpretabilidad**: Son "cajas negras" - difícil entender por qué toman ciertas decisiones. Esto complica la depuración y es problemático en aplicaciones críticas como medicina o justicia.
- **Overfitting**: Con datasets pequeños, pueden memorizar en lugar de generalizar. Técnicas como dropout y data augmentation mitigan esto, pero requieren expertise.
- **Sensibilidad a hiperparámetros**: El rendimiento depende de elecciones como tamaño de kernel, número de filtros y arquitectura. Encontrar la configuración óptima requiere experimentación.

Limitaciones

Más allá de las desventajas, las CNN tienen limitaciones inherentes:

- **Dependencia de datos locales**: Capturan bien patrones locales, pero dependencias a largo alcance (como en lenguaje) son mejor manejadas por transformers.
- **Invarianza excesiva**: Pueden perder información de posición precisa, problemático en tareas que requieren localización exacta.
- **Escalabilidad**: Redes muy profundas sufren vanishing gradient, aunque arquitecturas como ResNet lo resuelven parcialmente.

En comparación con transformers, las CNN son más eficientes computacionalmente pero menos flexibles para secuencias variables.

Comparación con Alternativas

- **Vs. Redes tradicionales**: CNN superan en precisión y automatización, pero requieren más datos.
- **Vs. Transformers**: Transformers son mejores para secuencias largas, pero CNN son más eficientes para datos locales.
- **Vs. Métodos clásicos**: SVM o random forests son más interpretables, pero CNN los superan en complejidad.

En resumen, las CNN son ideales para visión por computadora con datos abundantes, pero requieren consideración cuidadosa en otros contextos.

In [1]:
```
# Clasificación de dígitos con CNN (simple)

import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# 1. Cargar datos
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# 2. Preprocesamiento
x_train = x_train.reshape((60000, 28, 28, 1)) / 255.0
x_test = x_test.reshape((10000, 28, 28, 1)) / 255.0

# 3. Modelo CNN
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# 4. Compilar
```

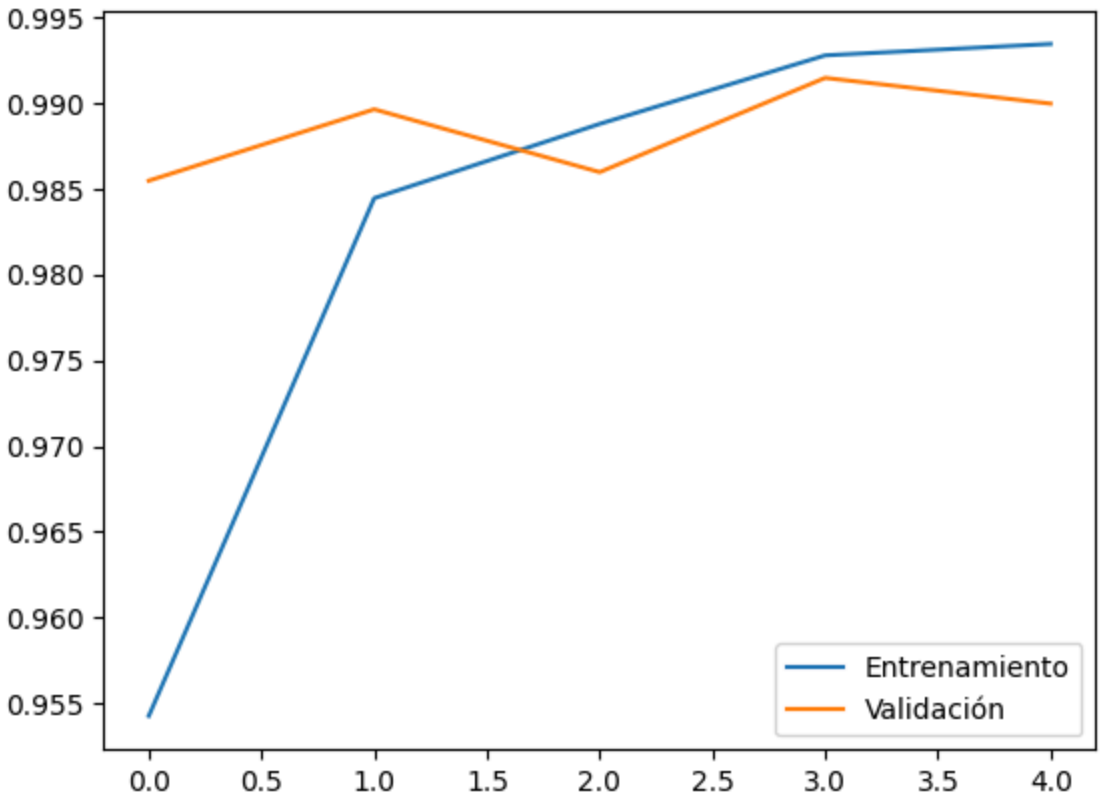
```python
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# 5. Entrenar
history = model.fit(x_train, y_train, epochs=5, validation_split=0.1)

# 6. Evaluar
test_loss, test_acc = model.evaluate(x_test, y_test)
print("Precisión:", test_acc)

# 7. Gráfica
plt.plot(history.history['accuracy'], label='Entrenamiento')
plt.plot(history.history['val_accuracy'], label='Validación')
plt.legend()
plt.show()
```

```
C:\Users\erick\anaconda3\envs\maquinas\Lib\site-packages\keras\src\layers\convolutio
nal\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument
to a layer. When using Sequential models, prefer using an `Input(shape)` object as t
he first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 28s 15ms/step - accuracy: 0.9543 - loss: 0.1505 - val
_accuracy: 0.9855 - val_loss: 0.0526
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 25s 15ms/step - accuracy: 0.9845 - loss: 0.0498 - val
_accuracy: 0.9897 - val_loss: 0.0380
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 15ms/step - accuracy: 0.9888 - loss: 0.0347 - val
_accuracy: 0.9860 - val_loss: 0.0457
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 15ms/step - accuracy: 0.9928 - loss: 0.0236 - val
_accuracy: 0.9915 - val_loss: 0.0320
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 41s 15ms/step - accuracy: 0.9935 - loss: 0.0196 - val
_accuracy: 0.9900 - val_loss: 0.0320
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9912 - loss: 0.0307
Precisión: 0.9911999702453613
```



8. Conclusiones

Transformar el campo de la visión por computadora y el aprendizaje profundo.

Hacer indispensables las CNN en aplicaciones modernas de IA.

Apuntar el futuro a arquitecturas híbridas que combinen CNN con Transformers.

9. Recomendaciones

Comenzar con datasets simples como MNIST.

Explorar arquitecturas modernas como EfficientNet.

Usar transfer learning con modelos pre-entrenados.

Recomendar estudiar tanto CNN como Transformers para una comprensión completa del deep learning moderno.

10. Referencias

1. LeCun, Y., et al. (1989). Backpropagation applied to handwritten zip code recognition. Neural Computation.

2. Krizhevsky, A., et al. (2012). ImageNet classification with deep convolutional neural networks. NIPS.
3. He, K., et al. (2016). Deep residual learning for image recognition. CVPR.
4. Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv.
5. Chollet, F. (2017). Deep Learning with Python. Manning Publications.
6. Goodfellow, I., et al. (2016). Deep Learning. MIT Press.

In [ ]: